Personalized Networking Assistant

Project Description:

Personalized Networking Assistant is an AI-powered application designed to help users build meaningful professional connections during networking events, conferences, seminars, workshops, and career fairs. The platform analyzes event descriptions and user interests to generate personalized conversation starters, identify relevant discussion topics, and provide verified information about those topics.
Built using **FastAPI**, **Streamlit**, **Natural Language Processing (NLP)**, and the **Wikipedia API**, the application enables users to prepare for networking opportunities with confidence. The system also maintains conversation history, collects user feedback, and generates downloadable PDF reports for future reference.

Scenarios

Scenario 1:

A university student attending an AI conference enters the event description and selects interests such as Artificial Intelligence, Machine Learning, and Startups. The system analyzes the event and generates personalized conversation starters to help the student interact with speakers and industry experts.

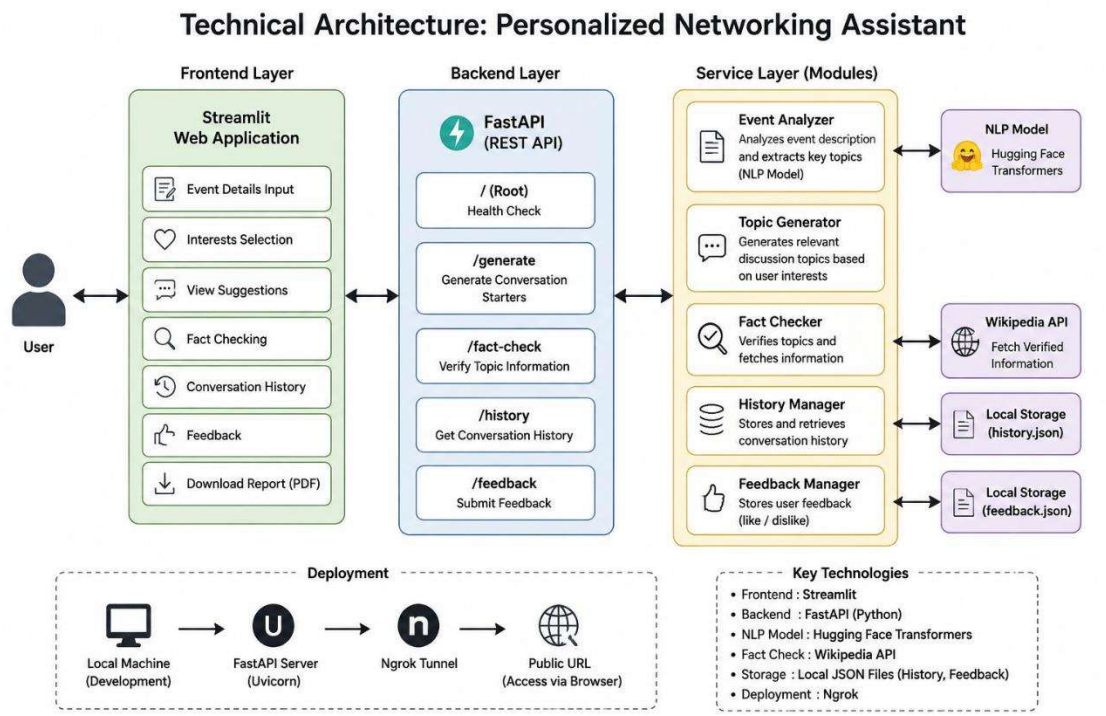
Scenario 2:

A job seeker participating in a career fair enters details about the event and selects interests related to Software Engineering and Data Science. The system suggests relevant discussion topics and conversation starters to help the user network with recruiters.

Scenario 3:

A professional attending a cloud computing summit enters the event details and receives AI-generated conversation suggestions, verified information about discussed technologies, and a downloadable networking preparation report.

Technical Architecture



Description:

The Personalized Networking Assistant follows a modular three-tier architecture.

- The **Frontend Layer** consists of a Streamlit-based user interface.
- The **Backend Layer** is implemented using FastAPI and handles request processing and API routing.
- The **Service Layer** contains reusable modules for event analysis, topic generation, fact verification, history logging, and feedback management.

The architecture enables scalability, maintainability, and modular development.

Pre-requisites

- Python Programming Knowledge
- FastAPI Framework Knowledge
- Streamlit Framework Knowledge
- Natural Language Processing Concepts
- Hugging Face Transformers Library
- REST API Concepts
- Git and GitHub Basics
- Virtual Environment Management

Project Workflow

Milestone 1: Environment Setup and Architecture Design

Activity 1.1: Define Project Requirements

- Identify user requirements.
- Define networking use cases.
- Finalize project scope.

Activity 1.2: Design System Architecture

- Create architectural diagrams.
- Define interactions between frontend, backend, and services.

Activity 1.3: Set Up Development Environment

- Install Python.
- Create virtual environment.
- Install dependencies.

Activity 1.4: Create Project Structure

- Create folders for:
  - app/
  - services/
  - models/
  - routers/
  - frontend/
  - tests/

Milestone 2: Backend Development

Activity 2.1: Implement FastAPI Backend

Create API endpoints:

- /
- /generate
- /fact-check
- /history

- /feedback

Activity 2.2: Develop Event Analysis Module

Implement NLP-based event analysis using Hugging Face Transformers.

Functions:

analyze_event()

Responsibilities:

- Analyze event descriptions.
- Detect important topics.

Activity 2.3: Develop Topic Generation Module

Functions:

generate_topics()

Responsibilities:

- Generate personalized discussion topics.
- Match topics with user interests.

Activity 2.4: Develop Fact Verification Module

Integrate Wikipedia API.

Functions:

verify_topic()

Responsibilities:

- Retrieve verified summaries.
- Validate discussion topics.

Milestone 3: Frontend Development

Activity 3.1: Develop Streamlit Interface

Create user-friendly pages for:

- Event input.
- Interest selection.
- Topic display.
- Suggestion display.
- Fact checking.

Activity 3.2: Add User Interaction Features

Implement:

- Like/Dislike Feedback.
- Conversation History.
- PDF Download.
- Loading animations.

Milestone 4: Data Storage and Logging

Activity 4.1: Conversation History

Store generated conversations in:

history.json
Responsibilities:

- Save generated suggestions.
- Retrieve historical conversations.

Activity 4.2: Feedback Collection
Store user ratings in:
feedback.json
Responsibilities:

- Save likes/dislikes.
- Analyze user preferences.

Milestone 5: Testing and Validation
Activity 5.1: Unit Testing
Create test files:
test_event_analyzer.py
test_fact_checker.py
test_routes.py
Use:
pytest -v
Validate:

- API functionality.
- NLP modules.
- Fact checking.
- Route responses.

Activity 5.2: Swagger API Testing
Access Swagger UI:
http://127.0.0.1:8000/docs
Test:

- Generate endpoint.
- Fact-check endpoint.
- History endpoint.

Milestone 6: Deployment and Documentation
Activity 6.1: Local Deployment
Run FastAPI:
python -m uvicorn app.main:app --reload
Run Streamlit:
streamlit run frontend/streamlit_app.py

Activity 6.2: Version Control
Upload project to GitHub.
Repository includes:

- Source code.
- Documentation.
- Images.
- Tests.

- README.

Exploring the Application Pages

Home Page

Description:

The homepage allows users to:

- Enter event descriptions.
- Specify interests.
- Generate networking suggestions.

Topics Section

Description:

Displays AI-detected topics extracted from the event description.

Example:

Artificial Intelligence

Machine Learning

Cloud Computing

Startups

Conversation Suggestions Section

Description:

Displays personalized conversation starters.

Example:

What inspired your interest in Generative AI?

How do you see AI impacting startups?

Fact Checker Page

Description:

Users can verify information related to any discussion topic using Wikipedia.

History Section

Description:

Displays previously generated networking sessions.

Conclusion

Personalized Networking Assistant successfully demonstrates how Artificial Intelligence and Natural Language Processing can improve professional networking experiences. The system assists users in preparing for networking events by generating personalized conversation starters, detecting relevant topics, verifying information, and maintaining interaction history.

The modular architecture built using FastAPI and Streamlit ensures maintainability and scalability. Future enhancements include cloud deployment, user authentication, database integration, multilingual support, and integration with advanced Large Language Models such as Gemini or GPT.

This project highlights the practical application of AI in enhancing communication, professional growth, and networking effectiveness.